

Comprehensive Guide to Modern Server Management & DevOps

1. Server Infrastructure & Architecture

1.1 Cloud Infrastructure Models

Multi-Cloud Architecture:

```yaml

# terraform/multi-cloud-setup.tf

```
terraform {
 required_version = ">= 1.0"
 required_providers {
 aws = {
 source = "hashicorp/aws"
 version = "~> 4.0"
 }
 google = {
 source = "hashicorp/google"
 version = "~> 4.0"
 }
 azurerm = {
 source = "hashicorp/azurerm"
 version = "~> 3.0"
 }
 }
}
```

# AWS EC2 Instances

```
resource "aws_instance" "web_servers" {
 count = 3
 ami = "ami-0c02fb55956c7d316"
 instance_type = "t3.medium"
 key_name = aws_key_pair.deployer.key_name

 tags = {
 Name = "web-server-${count.index}"
 Environment = "production"
 Role = "web"
 }

 user_data = file("scripts/web-server-setup.sh")
}
```

# Google Cloud Compute

```
resource "google_compute_instance" "api_servers" {
 count = 2
 name = "api-server-${count.index}"
 machine_type = "e2-medium"
 zone = "us-central1-a"

 boot_disk {
 initialize_params {
 image = "ubuntu-os-cloud/ubuntu-2004-lts"
 }
 }
}
```

```
network_interface {
 network = "default"
 access_config {}
}
```

```
metadata_startup_script = file("scripts/api-server-setup.sh")
}
```

# Azure Virtual Machines

```
resource "azurerm_linux_virtual_machine" "database_servers" {
 count = 2
 name = "db-server-${count.index}"
 resource_group_name = azurerm_resource_group.main.name
 location = azurerm_resource_group.main.location
 size = "Standard_B2s"
 admin_username = "adminuser"
```

```
 network_interface_ids = [
 azurerm_network_interface.main[count.index].id,
]
```

```
 admin_ssh_key {
 username = "adminuser"
 public_key = file("~/ssh/id_rsa.pub")
 }
```

```
 os_disk {
 caching = "ReadWrite"
 storage_account_type = "Standard_LRS"
```

```
}
```

```
source_image_reference {
 publisher = "Canonical"
 offer = "UbuntuServer"
 sku = "16.04-LTS"
 version = "latest"
}
```

```
}
```

```
...
```

```
1.2 High Availability Setup
```

```
```bash
```

```
#!/bin/bash
```

```
# ha-setup.sh - High Availability Load Balancer Configuration
```

```
# Install and configure HAProxy
```

```
apt-get update && apt-get install -y haproxy
```

```
# Configure HAProxy
```

```
cat > /etc/haproxy/haproxy.cfg << EOF
```

```
global
```

```
    daemon
```

```
    maxconn 4096
```

```
    user haproxy
```

```
    group haproxy
```

```
defaults
```

```
mode http
timeout connect 5000ms
timeout client 50000ms
timeout server 50000ms
option forwardfor
option http-server-close
```

```
frontend http_front
    bind *:80
    stats uri /haproxy?stats
    default_backend http_back
```

```
backend http_back
    balance roundrobin
    option httpchk GET /health
    server web01 10.0.1.10:80 check
    server web02 10.0.1.11:80 check
    server web03 10.0.1.12:80 check
```

```
frontend https_front
    bind *:443 ssl crt /etc/ssl/certs/mydomain.pem
    default_backend http_back
EOF
```

```
# Enable and start HAProxy
systemctl enable haproxy
systemctl start haproxy
```

```
# Configure keepalived for failover
```

```
apt-get install -y keepalived
```

```
cat > /etc/keepalived/keepalived.conf << EOF
```

```
vrrp_script chk_haproxy {  
    script "pidof haproxy"  
    interval 2  
    weight 2  
}
```

```
vrrp_instance VI_1 {  
    state MASTER  
    interface eth0  
    virtual_router_id 51  
    priority 101  
    advert_int 1
```

```
    authentication {  
        auth_type PASS  
        auth_pass 1111  
    }
```

```
    virtual_ipaddress {  
        10.0.1.100  
    }
```

```
    track_script {  
        chk_haproxy  
    }  
}
```

EOF

```
systemctl enable keepalived
```

```
systemctl start keepalived
```

```
...
```

```
---
```

2. Containerization & Orchestration

2.1 Docker & Container Management

```
``dockerfile
```

```
# Dockerfile - Multi-stage build for Node.js application
```

```
FROM node:18-alpine AS builder
```

```
WORKDIR /app
```

```
# Copy package files
```

```
COPY package*.json ./
```

```
COPY tsconfig.json ./
```

```
# Install dependencies
```

```
RUN npm ci --only=production && npm cache clean --force
```

```
# Copy source code
```

```
COPY src/ ./src/
```

```
# Build application
```

RUN npm run build

Production stage

FROM node:18-alpine AS production

RUN addgroup -g 1001 -S nodejs

RUN adduser -S nextjs -u 1001

WORKDIR /app

Copy built application

COPY --from=builder --chown=nextjs:nodejs /app/dist/ ./dist/

COPY --from=builder --chown=nextjs:nodejs /app/node_modules/ ./node_modules/

COPY --from=builder --chown=nextjs:nodejs /app/package.json ./

USER nextjs

EXPOSE 3000

ENV NODE_ENV=production

ENV PORT=3000

Health check

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \

CMD node healthcheck.js

CMD ["node", "dist/server.js"]

...


```
``yaml
```

```
# docker-compose.prod.yml
```

```
version: '3.8'
```

```
services:
```

```
  app:
```

```
    build:
```

```
      context: .
```

```
      target: production
```

```
    ports:
```

```
      - "3000:3000"
```

```
    environment:
```

```
      - NODE_ENV=production
```

```
      - DATABASE_URL=${DATABASE_URL}
```

```
      - REDIS_URL=${REDIS_URL}
```

```
    depends_on:
```

```
      - database
```

```
      - redis
```

```
    deploy:
```

```
      replicas: 3
```

```
      restart_policy:
```

```
        condition: on-failure
```

```
        delay: 5s
```

```
        max_attempts: 3
```

```
    resources:
```

```
      limits:
```

```
        memory: 512M
```

```
        cpus: '1.0'
```

```
      reservations:
```

memory: 256M

cpus: '0.5'

healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:3000/health"]

interval: 30s

timeout: 10s

retries: 3

database:

image: postgres:14-alpine

environment:

- POSTGRES_DB=\${DB_NAME}
- POSTGRES_USER=\${DB_USER}
- POSTGRES_PASSWORD=\${DB_PASSWORD}

volumes:

- postgres_data:/var/lib/postgresql/data
- ./init.sql:/docker-entrypoint-initdb.d/init.sql

deploy:

resources:

limits:

memory: 1G

cpus: '1.0'

redis:

image: redis:6-alpine

command: redis-server --appendonly yes

volumes:

- redis_data:/data

deploy:

```
resources:
  limits:
    memory: 256M
    cpus: '0.5'
```

```
nginx:
  image: nginx:alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
    - ./ssl:/etc/nginx/ssl
  depends_on:
    - app
  deploy:
    resources:
      limits:
        memory: 128M
        cpus: '0.25'
```

```
volumes:
  postgres_data:
  redis_data:
  ...
```

2.2 Kubernetes Orchestration

```yaml

```
k8s/app-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: web-app
 labels:
 app: web-app
 version: v1
spec:
 replicas: 3
 selector:
 matchLabels:
 app: web-app
 strategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1
 maxUnavailable: 0
 template:
 metadata:
 labels:
 app: web-app
 version: v1
 annotations:
 prometheus.io/scrape: "true"
 prometheus.io/port: "3000"
 prometheus.io/path: "/metrics"
 spec:
 containers:
```

```
- name: web-app
 image: myregistry/web-app:latest
 ports:
 - containerPort: 3000
 env:
 - name: NODE_ENV
 value: "production"
 - name: DATABASE_URL
 valueFrom:
 secretKeyRef:
 name: app-secrets
 key: database-url
 - name: REDIS_URL
 value: "redis://redis-service:6379"
 resources:
 requests:
 memory: "256Mi"
 cpu: "250m"
 limits:
 memory: "512Mi"
 cpu: "500m"
 livenessProbe:
 httpGet:
 path: /health
 port: 3000
 initialDelaySeconds: 30
 periodSeconds: 10
 timeoutSeconds: 5
 failureThreshold: 3
```

readinessProbe:

httpGet:

path: /ready

port: 3000

initialDelaySeconds: 5

periodSeconds: 5

timeoutSeconds: 3

startupProbe:

httpGet:

path: /health

port: 3000

initialDelaySeconds: 10

periodSeconds: 5

failureThreshold: 10

volumeMounts:

- name: config-volume

mountPath: /app/config

volumes:

- name: config-volume

configMap:

name: app-config

affinity:

podAntiAffinity:

preferredDuringSchedulingIgnoredDuringExecution:

- weight: 100

podAffinityTerm:

labelSelector:

matchExpressions:

- key: app

operator: In

values:

- web-app

topologyKey: kubernetes.io/hostname

---

apiVersion: v1

kind: Service

metadata:

name: web-app-service

spec:

selector:

app: web-app

ports:

- port: 80

targetPort: 3000

protocol: TCP

type: LoadBalancer

---

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: web-app-hpa

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: web-app

minReplicas: 2

maxReplicas: 10

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 70

- type: Resource

resource:

name: memory

target:

type: Utilization

averageUtilization: 80

...

```yaml

k8s/ingress.yaml

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: main-ingress

annotations:

kubernetes.io/ingress.class: "nginx"

nginx.ingress.kubernetes.io/rewrite-target: /

nginx.ingress.kubernetes.io/ssl-redirect: "true"

cert-manager.io/cluster-issuer: "letsencrypt-prod"

spec:

tls:

- hosts:

- app.example.com

secretName: app-tls-secret

rules:

- host: app.example.com

http:

paths:

- path: /

pathType: Prefix

backend:

service:

name: web-app-service

port:

number: 80

- path: /api

pathType: Prefix

backend:

service:

name: api-service

port:

number: 8080

...

3. Infrastructure as Code (IaC)

3.1 Terraform Modules

```hcl

```
modules/ec2-instance/main.tf
```

```
variable "name" {
 description = "Instance name"
 type = string
}
```

```
variable "instance_type" {
 description = "EC2 instance type"
 type = string
 default = "t3.micro"
}
```

```
variable "vpc_id" {
 description = "VPC ID"
 type = string
}
```

```
variable "subnet_id" {
 description = "Subnet ID"
 type = string
}
```

```
variable "security_group_ids" {
 description = "List of security group IDs"
 type = list(string)
}
```

```
variable "user_data" {
 description = "User data script"
```

```
type = string
default = ""
}
```

```
resource "aws_instance" "this" {
 ami = data.aws_ami.ubuntu.id
 instance_type = var.instance_type
 subnet_id = var.subnet_id
 vpc_security_group_ids = var.security_group_ids
 key_name = aws_key_pair.this.key_name
```

```
 user_data = var.user_data
```

```
 root_block_device {
 volume_type = "gp3"
 volume_size = 20
 encrypted = true
 }
```

```
 tags = {
 Name = var.name
 Environment = "production"
 ManagedBy = "terraform"
 }
```

```
 lifecycle {
 create_before_destroy = true
 ignore_changes = [ami]
 }
```

```
}
```

```
resource "aws_cloudwatch_metric_alarm" "cpu_high" {
 alarm_name = "${var.name}-cpu-utilization-high"
 comparison_operator = "GreaterThanThreshold"
 evaluation_periods = "2"
 metric_name = "CPUUtilization"
 namespace = "AWS/EC2"
 period = "120"
 statistic = "Average"
 threshold = "80"
 alarm_description = "This metric monitors EC2 CPU utilization"
 alarm_actions = [aws_sns_topic.alerts.arn]
```

```
 dimensions = {
 InstanceId = aws_instance.this.id
 }
}
```

```
data "aws_ami" "ubuntu" {
 most_recent = true
 owners = ["099720109477"] # Canonical
```

```
 filter {
 name = "name"
 values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
 }
```

```
 filter {
```

```
 name = "virtualization-type"
 values = ["hvm"]
 }
}
...

```

### ### \*\*3.2 Ansible Configuration Management\*\*

```
``yaml
ansible/playbooks/setup-web-server.yml

- name: Configure production web servers
 hosts: web_servers
 become: yes
 vars:
 app_user: "deployer"
 app_group: "deployer"
 app_directory: "/opt/myapp"
 node_version: "18.x"

 pre_tasks:
 - name: Update apt cache
 apt:
 update_cache: yes
 cache_valid_time: 3600
 when: ansible_os_family == "Debian"

 roles:
 - role: common

```

- role: nodejs
- role: nginx
- role: app\_deployment

tasks:

- name: Create application user

user:

name: "{{ app\_user }}"  
group: "{{ app\_group }}"  
shell: /bin/bash  
home: "/home/{{ app\_user }}"  
create\_home: yes  
state: present

- name: Create application directory

file:

path: "{{ app\_directory }}"  
state: directory  
owner: "{{ app\_user }}"  
group: "{{ app\_group }}"  
mode: '0755'

- name: Install Node.js

include\_role:

name: nodejs

vars:

nodejs\_version: "{{ node\_version }}"

- name: Configure nginx

template:

src: nginx.conf.j2

dest: /etc/nginx/sites-available/{{ inventory\_hostname }}

owner: root

group: root

mode: '0644'

notify: restart nginx

- name: Enable nginx site

file:

src: /etc/nginx/sites-available/{{ inventory\_hostname }}

dest: /etc/nginx/sites-enabled/{{ inventory\_hostname }}

state: link

notify: restart nginx

- name: Setup SSL with Certbot

include\_role:

name: nginx

vars:

nginx\_ssl: true

nginx\_ssl\_certificate: /etc/letsencrypt/live/{{ domain }}/fullchain.pem

nginx\_ssl\_certificate\_key: /etc/letsencrypt/live/{{ domain }}/privkey.pem

- name: Configure firewall

ufw:

rule: "{{ item.rule }}"

port: "{{ item.port }}"

proto: "{{ item.proto | default('tcp') }}"

loop:

- { rule: allow, port: '22', proto: 'tcp' }
- { rule: allow, port: '80', proto: 'tcp' }
- { rule: allow, port: '443', proto: 'tcp' }

- name: Deploy application

include\_role:

name: app\_deployment

vars:

app\_source: "{{ build\_artifact\_url }}"

app\_env: "production"

- name: Setup monitoring

include\_role:

name: monitoring

vars:

monitoring\_agent: "datadog"

datadog\_api\_key: "{{ datadog\_api\_key }}"

handlers:

- name: restart nginx

service:

name: nginx

state: restarted

- name: restart app

systemd:

name: myapp

state: restarted

daemon\_reload: yes



...

---

## \*\*4. Monitoring & Observability\*\*

### \*\*4.1 Comprehensive Monitoring Stack\*\*

``yaml

# docker-compose.monitoring.yml

version: '3.8'

services:

prometheus:

image: prom/prometheus:latest

ports:

- "9090:9090"

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

- prometheus\_data:/prometheus

command:

- '--config.file=/etc/prometheus/prometheus.yml'

- '--storage.tsdb.path=/prometheus'

- '--web.console.libraries=/etc/prometheus/console\_libraries'

- '--web.console.templates=/etc/prometheus/consoles'

- '--storage.tsdb.retention.time=200h'

- '--web.enable-lifecycle'

deploy:

resources:

limits:

memory: 2G

cpus: '1.0'

grafana:

image: grafana/grafana:latest

ports:

- "3000:3000"

environment:

- GF\_SECURITY\_ADMIN\_PASSWORD=admin123

- GF\_USERS\_ALLOW\_SIGN\_UP=false

volumes:

- grafana\_data:/var/lib/grafana

- ./grafana/dashboards:/etc/grafana/provisioning/dashboards

- ./grafana/datasources:/etc/grafana/provisioning/datasources

depends\_on:

- prometheus

alertmanager:

image: prom/alertmanager:latest

ports:

- "9093:9093"

volumes:

- ./alertmanager.yml:/etc/alertmanager/alertmanager.yml

command:

- '--config.file=/etc/alertmanager/alertmanager.yml'

- '--storage.path=/alertmanager'

node-exporter:

image: prom/node-exporter:latest

ports:

- "9100:9100"

volumes:

- /proc:/host/proc:ro

- /sys:/host/sys:ro

- /:/rootfs:ro

command:

- '--path.procfs=/host/proc'

- '--path.rootfs=/rootfs'

- '--path.sysfs=/host/sys'

- '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)(\$|/)'

cadvisor:

image: gcr.io/cadvisor/cadvisor:latest

ports:

- "8080:8080"

volumes:

- /:/rootfs:ro

- /var/run:/var/run:rw

- /sys:/sys:ro

- /var/lib/docker:/var/lib/docker:ro

volumes:

prometheus\_data:

grafana\_data:

...

```yaml

```
# prometheus.yml

global:

  scrape_interval: 15s

  evaluation_interval: 15s


rule_files:

- "alert_rules.yml"


alerting:

  alertmanagers:

    - static_configs:

        - targets:

            - alertmanager:9093


scrape_configs:

- job_name: 'prometheus'

  static_configs:

    - targets: ['localhost:9090']


- job_name: 'node-exporter'

  static_configs:

    - targets: ['node-exporter:9100']


- job_name: 'cadvisor'

  static_configs:

    - targets: ['cadvisor:8080']


- job_name: 'web-app'

  static_configs:
```

```

- targets: ['web-app:3000']

metrics_path: /metrics

scrape_interval: 10s


- job_name: 'nginx'
  static_configs:
    - targets: ['nginx:9113']
...


### **4.2 Custom Application Metrics**


```python
monitoring/metrics.py

from prometheus_client import Counter, Histogram, Gauge, generate_latest
import time
import logging

Application metrics
REQUEST_COUNT = Counter(
 'http_requests_total',
 'Total HTTP requests',
 ['method', 'endpoint', 'status']
)

REQUEST_DURATION = Histogram(
 'http_request_duration_seconds',
 'HTTP request duration',
 ['method', 'endpoint']
)

```

```
ACTIVE_USERS = Gauge(
 'active_users_total',
 'Number of active users'
)
```

```
DATABASE_CONNECTIONS = Gauge(
 'database_connections_total',
 'Number of active database connections'
)
```

```
ERROR_COUNT = Counter(
 'application_errors_total',
 'Total application errors',
 ['error_type']
)
```

```
CACHE_HITS = Counter(
 'cache_hits_total',
 'Total cache hits',
 ['cache_type']
)
```

```
CACHE_MISSES = Counter(
 'cache_misses_total',
 'Total cache misses',
 ['cache_type']
)
```

```
Middleware for request tracking

async def metrics_middleware(request, call_next):

 start_time = time.time()

 try:

 response = await call_next(request)

 # Record metrics

 REQUEST_COUNT.labels(

 method=request.method,

 endpoint=request.url.path,

 status=response.status_code

).inc()

 except Exception as e:

 ERROR_COUNT.labels(error_type=type(e).__name__).inc()

 raise

 finally:

 duration = time.time() - start_time

 REQUEST_DURATION.labels(

 method=request.method,

 endpoint=request.url.path

).observe(duration)

Database monitoring

class DatabaseMonitor:
```

```
def __init__(self, db_pool):
```

```
 self.db_pool = db_pool
```

```
async def collect_metrics(self):
```

```
 try:
```

```
 # Get connection count
```

```
 async with self.db_pool.acquire() as conn:
```

```
 result = await conn.fetchval("SELECT count(*) FROM pg_stat_activity")
```

```
 DATABASE_CONNECTIONS.set(result)
```

```
 except Exception as e:
```

```
 logging.error(f"Failed to collect database metrics: {e}")
```

```
Cache monitoring
```

```
class CacheMonitor:
```

```
 def __init__(self, redis_client):
```

```
 self.redis = redis_client
```

```
async def record_hit(self, cache_type):
```

```
 CACHE_HITS.labels(cache_type=cache_type).inc()
```

```
async def record_miss(self, cache_type):
```

```
 CACHE_MISSES.labels(cache_type=cache_type).inc()
```

```
...
```

```

```

```
5. Security & Hardening
```

```
5.1 Server Security Configuration
```



```
```bash
```

```
#!/bin/bash
```

```
# server-hardening.sh
```

```
# Update system
```

```
apt-get update && apt-get upgrade -y
```

```
# Install essential security packages
```

```
apt-get install -y \
```

```
fail2ban \
```

```
ufw \
```

```
unattended-upgrades \
```

```
apt-listchanges \
```

```
lynis
```

```
# Configure firewall
```

```
ufw default deny incoming
```

```
ufw default allow outgoing
```

```
ufw allow ssh
```

```
ufw allow http
```

```
ufw allow https
```

```
ufw --force enable
```

```
# Configure fail2ban
```

```
cat > /etc/fail2ban/jail.local << EOF
```

```
[DEFAULT]
```

```
bantime = 3600
```

```
findtime = 600
```

```
maxretry = 3
```

```
[sshd]
```

```
enabled = true
```

```
port = ssh
```

```
logpath = /var/log/auth.log
```

```
maxretry = 3
```

```
EOF
```

```
systemctl enable fail2ban
```

```
systemctl start fail2ban
```

```
# Configure automatic security updates
```

```
cat > /etc/apt/apt.conf.d/50unattended-upgrades << EOF
```

```
Unattended-Upgrade::Allowed-Origins {
```

```
    "${distro_id}:${distro_codename}";
```

```
    "${distro_id}:${distro_codename}-security";
```

```
    "${distro_id}ESM:${distro_codename}";
```

```
};
```

```
Unattended-Upgrade::AutoFixInterruptedDpkg "true";
```

```
Unattended-Upgrade::MinimalSteps "true";
```

```
Unattended-Upgrade::Remove-Unused-Dependencies "true";
```

```
Unattended-Upgrade::Automatic-Reboot "true";
```

```
Unattended-Upgrade::Automatic-Reboot-Time "02:00";
```

```
EOF
```

```
# SSH hardening
```

```
sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
```

```
sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin no/' /etc/ssh/sshd_config
```

```
sed -i 's/X11Forwarding yes/X11Forwarding no/' /etc/ssh/sshd_config
echo "AllowUsers deployer" >> /etc/ssh/sshd_config
```

System limits

```
echo "* hard core 0" >> /etc/security/limits.conf
echo "fs.suid_dumpable = 0" >> /etc/sysctl.conf
```

Configure auditd

```
apt-get install -y auditd
cat > /etc/audit/rules.d/hardening.rules << EOF
-w /etc/passwd -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/sudoers -p wa -k identity
-w /var/log/auth.log -p wa -k authentication
-w /var/log/faillog -p wa -k authentication
EOF
```

```
systemctl enable auditd
systemctl start auditd
```

File integrity monitoring

```
apt-get install -y aide
aideinit
mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

```
echo "0 2 * * * /usr/bin/aide --check" | crontab -
```

Security scanning with Lynis

lynis audit system

```

### \*\*5.2 Kubernetes Security\*\*

```yaml

k8s/security/psp.yaml

apiVersion: policy/v1beta1

kind: PodSecurityPolicy

metadata:

name: restricted

spec:

privileged: false

allowPrivilegeEscalation: false

requiredDropCapabilities:

- ALL

volumes:

- 'configMap'

- 'emptyDir'

- 'projected'

- 'secret'

- 'downwardAPI'

- 'persistentVolumeClaim'

hostNetwork: false

hostIPC: false

hostPID: false

runAsUser:

rule: 'MustRunAsNonRoot'

seLinux:

```

    rule: 'RunAsAny'
  fsGroup:
    rule: 'RunAsAny'
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: psp:restricted
rules:
- apiGroups: ['policy']
  resources: ['podsecuritypolicies']
  verbs: ['use']
  resourceNames: ['restricted']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: psp:restricted
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: psp:restricted
subjects:
- kind: Group
  name: system:authenticated
  apiGroup: rbac.authorization.k8s.io
...

```

```
``yaml
```

```
# k8s/security/network-policies.yaml
```

```
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy
```

```
metadata:
```

```
  name: deny-all
```

```
spec:
```

```
  podSelector: {}
```

```
  policyTypes:
```

```
  - Ingress
```

```
  - Egress
```

```
---
```

```
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy
```

```
metadata:
```

```
  name: web-app-allow
```

```
spec:
```

```
  podSelector:
```

```
    matchLabels:
```

```
      app: web-app
```

```
  policyTypes:
```

```
  - Ingress
```

```
  - Egress
```

```
  ingress:
```

```
  - from:
```

```
    - namespaceSelector:
```

```
      matchLabels:
```

```
        name: ingress-nginx
```

```
  ports:
```

```
  - protocol: TCP
```

```
    port: 3000
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
      ports:
      - protocol: TCP
        port: 5432
    - to:
    - podSelector:
        matchLabels:
          app: redis
      ports:
      - protocol: TCP
        port: 6379
    - to:
    - namespaceSelector:
        matchLabels:
          name: monitoring
      ports:
      - protocol: TCP
        port: 9090
  ...

---
```

6. Backup & Disaster Recovery

6.1 Automated Backup System

```
```bash
```

```
#!/bin/bash
```

```
backup-manager.sh
```

```
set -e
```

```
Configuration
```

```
BACKUP_DIR="/backups"
```

```
DATE=$(date +%Y%m%d_%H%M%S)
```

```
RETENTION_DAYS=7
```

```
S3_BUCKET="my-backup-bucket"
```

```
Database backup function
```

```
backup_database() {
```

```
 local db_name="$1"
```

```
 local backup_file="$BACKUP_DIR/${db_name}_${DATE}.sql.gz"
```

```
 echo "Backing up database: $db_name"
```

```
 pg_dump "$db_name" | gzip > "$backup_file"
```

```
Verify backup
```

```
if [$? -eq 0] && [-f "$backup_file"]; then
```

```
 echo "Database backup completed: $backup_file"
```

```
Upload to S3
```

```
aws s3 cp "$backup_file" "s3://$S3_BUCKET/databases/"
```



```

 # Local encryption (optional)

 gpg --encrypt --recipient "backup@company.com" "$backup_file"

 rm "$backup_file" # Remove unencrypted version
else
 echo "Database backup failed!"

 exit 1
fi
}

File system backup
backup_files() {
 local source_dir="$1"
 local backup_name="$2"
 local backup_file="$BACKUP_DIR/${backup_name}_${DATE}.tar.gz"

 echo "Backing up files: $source_dir"

 tar -czf "$backup_file" "$source_dir"

 if [$? -eq 0]; then
 echo "File backup completed: $backup_file"
 aws s3 cp "$backup_file" "s3://$S3_BUCKET/files/"
 else
 echo "File backup failed!"

 exit 1
 fi
}

```

```

Kubernetes backup

backup_kubernetes() {
 local namespace="$1"
 local backup_file="$BACKUP_DIR/k8s_${namespace}_${DATE}.tar.gz"

 echo "Backing up Kubernetes namespace: $namespace"

 # Backup resources
 kubectl get all -n "$namespace" -o yaml > "/tmp/${namespace}_resources.yaml"
 kubectl get configmaps -n "$namespace" -o yaml > "/tmp/${namespace}_configmaps.yaml"
 kubectl get secrets -n "$namespace" -o yaml > "/tmp/${namespace}_secrets.yaml"
 kubectl get pvc -n "$namespace" -o yaml > "/tmp/${namespace}_pvc.yaml"

 tar -czf "$backup_file" -C /tmp "${namespace}"_*.yaml

 if [$? -eq 0]; then
 echo "Kubernetes backup completed: $backup_file"
 aws s3 cp "$backup_file" "s3://$S3_BUCKET/kubernetes/"
 rm "/tmp/${namespace}"_*.yaml
 else
 echo "Kubernetes backup failed!"
 exit 1
 fi
}

Cleanup old backups

cleanup_backups() {
 echo "Cleaning up backups older than $RETENTION_DAYS days"

```

```

Local cleanup

find "$BACKUP_DIR" -name "*.gz" -mtime +$RETENTION_DAYS -delete
find "$BACKUP_DIR" -name "*.gpg" -mtime +$RETENTION_DAYS -delete

S3 cleanup (using lifecycle policies is better)
aws s3 ls "s3://$S3_BUCKET/" --recursive | \
while read -r line; do
 create_date=$(echo $line | awk '{print $1}')
 create_time=$(echo $line | awk '{print $2}')
 file_path=$(echo $line | awk '{print $4}')

 if [[-n $file_path]]; then
 file_date=$(date -d "$create_date $create_time" +%s)
 current_date=$(date +%s)
 difference=$(((current_date - file_date) / 86400))

 if [$difference -gt $RETENTION_DAYS]; then
 aws s3 rm "s3://$S3_BUCKET/$file_path"
 fi
 fi
done
}

Main execution
main() {
 # Create backup directory
 mkdir -p "$BACKUP_DIR"

 # Perform backups

```

```
backup_database "myapp_production"
backup_files "/opt/myapp" "application"
backup_files "/etc" "config"
backup_kubernetes "production"
```

```
Cleanup
```

```
cleanup_backups
```

```
Send notification
```

```
echo "Backup completed successfully" | mail -s "Backup Report" admin@company.com
```

```
echo "Backup process completed: $(date)"
```

```
}
```

```
Error handling
```

```
trap 'echo "Backup failed at line $LINENO"; exit 1' ERR
```

```
main "$@"
```

```
...
```

```
6.2 Disaster Recovery Plan
```

```
``yaml
```

```
dr-plan.yaml
```

```
version: '1.0'
```

```
plan_name: 'Production Environment DRP'
```

```
last_updated: '2024-01-15'
```

```
recovery_objectives:
```

rto: '4 hours' # Recovery Time Objective

rpo: '15 minutes' # Recovery Point Objective

teams:

infrastructure:

lead: 'infra-team@company.com'

backup: 'infra-backup@company.com'

database:

lead: 'dba@company.com'

backup: 'dba-backup@company.com'

application:

lead: 'app-team@company.com'

backup: 'app-backup@company.com'

recovery\_steps:

- phase: 'Assessment'

steps:

- 'Declare disaster (CTO/Infra Lead)'

- 'Activate DR team'

- 'Assess impact and scope'

- phase: